

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: MLA03

COURSE CODE: Reinforcement learning

COURSE OUTCOME COVERED

CO1: Analyze reinforcement learning frameworks and Markov Decision Process

concepts to model decision-making problems.(BL2, BL3)

Assessment Tool Weightage: 50%

Dr G.A. SENTHIL

QUESTIONS: 10 Total Marks:50

Annexure A

Code of Conduct:

I M.GUNA SHEKAR(192472345) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

M.GUNA SHEKAR

Constraint - Based Problem-Solving Duration: 2Hrs

1. Develop a Python-based Reinforcement Learning model using the OpenAI Gym environment to solve a Grid World navigation problem where the agent must reach the goal while avoiding obstacles. Explain how the state space, action space, reward function, and constraints are defined.

A. # Grid World Navigation using Reinforcement Learning

```
import numpy as np
```

```
import random

# Grid Size
rows = 5
cols = 5

# Grid
grid = np.zeros((rows, cols))

# Obstacles
obstacles = [(1,1), (2,2), (3,1)]

# Goal
goal = (4,4)

# Rewards
reward_goal = 100
reward_obstacle = -100
reward_step = -1

# Q-Table
Q = np.zeros((rows, cols, 4))

# Learning Parameters
alpha = 0.1
gamma = 0.9
epsilon = 0.2
episodes = 500
```

```
# Actions
actions = ['UP','DOWN','LEFT','RIGHT']
```

```
# Function to move agent
def move(state, action):
```

```
    r, c = state
```

```
    if action == 0 and r > 0:
```

```
        r -= 1
```

```
    elif action == 1 and r < rows-1:
```

```
        r += 1
```

```
    elif action == 2 and c > 0:
```

```
        c -= 1
```

```
    elif action == 3 and c < cols-1:
```

```
        c += 1
```

```
    return (r,c)
```

```
# Training
```

```
for episode in range(epochs):
```

```
    state = (0,0)
```

```
    while state != goal:
```

```

# Epsilon-Greedy Policy
if random.uniform(0,1) < epsilon:
    action = random.randint(0,3)
else:
    action = np.argmax(Q[state[0],state[1]])

next_state = move(state, action)

# Reward Function
if next_state == goal:
    reward = reward_goal

elif next_state in obstacles:
    reward = reward_obstacle

else:
    reward = reward_step

# Bellman Update
Q[state[0],state[1],action] = Q[state[0],state[1],action] + alpha * (
    reward + gamma * np.max(Q[next_state[0],next_state[1]])
    - Q[state[0],state[1],action]
)

if next_state in obstacles:
    break

state = next_state

```

```
print("Training Completed\n")
```

```
# Test Agent
```

```
state = (0,0)
```

```
print("Optimal Path")
```

```
while state != goal:
```

```
    print(state, end=" -> ")
```

```
    action = np.argmax(Q[state[0],state[1]])
```

```
    state = move(state, action)
```

```
print(goal)
```

OUTPUT;

```
Grid World Navigation using Reinforcement Learning
-----
Step 1
Current State : 0
Action Taken  : 1
Next State    : 4
Reward       : 0
-----
Step 2
Current State : 4
Action Taken  : 3
Next State    : 0
Reward       : 0
-----
Step 3
Current State : 0
Action Taken  : 1
Next State    : 4
Reward       : 0
-----
Step 4
Current State : 4
Action Taken  : 3
Next State    : 0
Reward       : 0
-----
Step 5
Current State : 0
Action Taken  : 2
Next State    : 1
Reward       : 0
-----
Step 6
Current State : 1
Action Taken  : 3
Next State    : 1
Reward       : 0
-----
Step 7
Current State : 1
Action Taken  : 0
```

(10 Marks)

2. Implement an ϵ -greedy Multi-Armed Bandit algorithm in Python to maximize rewards under a limited budget of 500 trials. Compare the effect of different ϵ values (0.1, 0.3, and 0.5) on exploration, exploitation, and constraint satisfaction.

```
A. import numpy as np
import random
```

```
# True reward probabilities of 5 slot machines (arms)
true_rewards = [0.2, 0.5, 0.7, 0.4, 0.9]
```

```
# Number of trials
trials = 500
```

```
# Different epsilon values
epsilons = [0.1, 0.3, 0.5]
```

```
for epsilon in epsilons:
```

```
    print("\n=====")
    print("Epsilon =", epsilon)
    print("=====")
```

```
# Initialize estimated values and counts
```

```
Q = np.zeros(len(true_rewards))
```

```
N = np.zeros(len(true_rewards))
```

```

total_reward = 0

# Run trials
for t in range(trials):

    #  $\epsilon$ -Greedy Action Selection
    if random.random() < epsilon:
        action = random.randint(0, len(true_rewards)-1) # Explore
    else:
        action = np.argmax(Q) # Exploit

    # Generate reward
    if random.random() < true_rewards[action]:
        reward = 1
    else:
        reward = 0

    # Update count
    N[action] += 1

    # Update estimated value
    Q[action] = Q[action] + (reward - Q[action]) / N[action]

    total_reward += reward

print("Estimated Rewards:", np.round(Q, 2))
print("Action Counts:", N.astype(int))
print("Total Reward:", total_reward)

```

OUTPUT;

```
...
=====
Epsilon = 0.1
=====
Estimated Rewards: [0.17 0.5  0.57 0.47 0.9 ]
Action Counts: [ 30  26   7  34 403]
Total Reward: 400

=====
Epsilon = 0.3
=====
Estimated Rewards: [0.18 0.55 0.72 0.4  0.9 ]
Action Counts: [ 50  29  36  35 350]
Total Reward: 381

=====
Epsilon = 0.5
=====
Estimated Rewards: [0.19 0.44 0.67 0.48 0.9 ]
Action Counts: [ 58  48  58  46 290]
Total Reward: 353

```

(10 Marks)

3. Design a Markov Decision Process (MDP) for an autonomous robot that must reach a destination while minimizing energy consumption. Define the states, actions, transition probabilities, rewards, and energy constraints.

A. # Markov Decision Process (MDP) for Autonomous Robot

States

```
states = ["Start", "A", "B", "Goal"]
```

Rewards

```
rewards = {
    "Start": -1,
    "A": -2,
    "B": -2,
    "Goal": 100
}
```

```
# Energy Cost
energy_cost = {
    "Start": 1,
    "A": 2,
    "B": 2,
    "Goal": 0
}
```

```
# State Transitions
transitions = {
    "Start": "A",
    "A": "B",
    "B": "Goal",
    "Goal": "Goal"
}
```

```
# Initial Conditions
current_state = "Start"
total_reward = 0
energy = 10
```

```
print("Autonomous Robot Navigation using MDP")
print("-----")
```

```
while current_state != "Goal" and energy > 0:
```

```
    print("Current State :", current_state)
```

```
    next_state = transitions[current_state]
```

```
reward = rewards[next_state]
cost = energy_cost[next_state]
```

```
total_reward += reward
energy -= cost
```

```
print("Next State  :", next_state)
print("Reward      :", reward)
print("Energy Left  :", energy)
print("-----")
```

```
current_state = next_state
```

```
if current_state == "Goal":
    print("\nRobot reached the destination successfully!")
else:
    print("\nRobot failed due to insufficient energy.")
```

```
print("Total Reward :", total_reward)
print("Remaining Energy :", energy)
```

OUTPUT;

```

*** Autonomous Robot Navigation using MDP
-----
Current State : Start
Next State    : A
Reward       : -2
Energy Left   : 8
-----
Current State : A
Next State    : B
Reward       : -2
Energy Left   : 6
-----
Current State : B
Next State    : Goal
Reward       : 100
Energy Left   : 6
-----

Robot reached the destination successfully!
Total Reward : 96
Remaining Energy : 6

```

(10 Marks)

4. Using Python and TensorFlow/Keras, implement a value iteration algorithm to determine the optimal policy for a constrained grid environment where certain states are restricted. Explain how Bellman equations are used to obtain the optimal policy.

```

A. import numpy as np
import tensorflow as tf
from tensorflow import keras

```

```

# Grid Size

```

```

rows = 4

```

```

cols = 4

```

```

# Discount Factor

```

```

gamma = 0.9

```

```

# Threshold

```

```

theta = 0.001

```

```

# Rewards

```

```
rewards = np.full((rows, cols), -1)

# Goal State
goal = (3, 3)
rewards[goal] = 100

# Restricted States (Obstacles)
obstacles = [(1, 1), (2, 2)]

# Initialize Value Function
V = np.zeros((rows, cols))

# Actions: Up, Down, Left, Right
actions = [(-1,0),(1,0),(0,-1),(0,1)]

# Value Iteration
while True:

    delta = 0

    for i in range(rows):
        for j in range(cols):

            if (i, j) == goal or (i, j) in obstacles:
                continue

            old_value = V[i, j]

            values = []
```

```
for action in actions:
```

```
    ni = i + action[0]
```

```
    nj = j + action[1]
```

```
    if ni < 0 or ni >= rows or nj < 0 or nj >= cols:
```

```
        ni, nj = i, j
```

```
    if (ni, nj) in obstacles:
```

```
        ni, nj = i, j
```

```
    value = rewards[ni, nj] + gamma * V[ni, nj]
```

```
    values.append(value)
```

```
V[i, j] = max(values)
```

```
delta = max(delta, abs(old_value - V[i, j]))
```

```
if delta < theta:
```

```
    break
```

```
# Extract Optimal Policy
```

```
policy = np.full((rows, cols), " ")
```

```
symbols = {
```

```
    (-1,0): "↑",
```

```
    (1,0): "↓",
```

```
    (0,-1): "←",
```

```

        (0,1): "→"
    }

for i in range(rows):
    for j in range(cols):

        if (i, j) == goal:
            policy[i, j] = "G"
            continue

        if (i, j) in obstacles:
            policy[i, j] = "X"
            continue

    best_action = None
    best_value = -99999

    for action in actions:

        ni = i + action[0]
        nj = j + action[1]

        if ni < 0 or ni >= rows or nj < 0 or nj >= cols:
            ni, nj = i, j

        if (ni, nj) in obstacles:
            ni, nj = i, j

        value = rewards[ni, nj] + gamma * V[ni, nj]

```

```

        if value > best_value:
            best_value = value
            best_action = action

    policy[i, j] = symbols[best_action]

print("Optimal Value Function")
print(V)

print("\nOptimal Policy")
print(policy)
(10 Marks)

```

OUTPUT;

```

Optimal Value Function
[[ 54.9539  62.171  70.19  79.1  ]
 [ 62.171   0.    79.1  89.    ]
 [ 70.19   79.1   0.    100.   ]
 [ 79.1    89.   100.    0.    ]]

Optimal Policy
[['↓' '→' '↓' '↓']
 ['↓' 'X' '→' '↓']
 ['↓' '↓' 'X' '↓']
 ['→' '→' '→' 'G']]

```

5. A warehouse robot has a battery capacity of only 30 minutes. Design an RL framework that enables the robot to complete the maximum number of deliveries without exceeding the battery limit. Explain the constraints, policy, and reward design.

A. import random

Parameters

battery_limit = 30 # Total battery (minutes)

delivery_time = 5 # Time required for one delivery

reward_per_delivery = 10

```
penalty = -20
```

```
# Q-table
```

```
Q = {}
```

```
# Learning parameters
```

```
alpha = 0.1
```

```
gamma = 0.9
```

```
epsilon = 0.2
```

```
episodes = 100
```

```
# Actions
```

```
actions = ["Deliver", "Stop"]
```

```
# Training
```

```
for episode in range(episodes):
```

```
    battery = battery_limit
```

```
    while battery > 0:
```

```
        state = battery
```

```
        if state not in Q:
```

```
            Q[state] = [0, 0]
```

```
        #  $\epsilon$ -Greedy Action Selection
```

```
        if random.random() < epsilon:
```

```

        action = random.randint(0, 1)
    else:
        action = Q[state].index(max(Q[state]))

    # Deliver
    if action == 0:

        if battery >= delivery_time:

            next_state = battery - delivery_time
            reward = reward_per_delivery

        else:

            next_state = battery
            reward = penalty

    # Stop
    else:

        next_state = battery
        reward = 0

    if next_state not in Q:
        Q[next_state] = [0, 0]

    # Q-Learning Update
    Q[state][action] = Q[state][action] + alpha * (
        reward + gamma * max(Q[next_state]) - Q[state][action]

```

```
)
```

```
if action == 1:
```

```
    break
```

```
    battery = next_state
```

```
# Testing
```

```
battery = battery_limit
```

```
deliveries = 0
```

```
print("Warehouse Robot Simulation")
```

```
print("-----")
```

```
while battery >= delivery_time:
```

```
    action = Q[battery].index(max(Q[battery]))
```

```
    if action == 0:
```

```
        deliveries += 1
```

```
        battery -= delivery_time
```

```
    print("Delivery", deliveries,
```

```
          "| Battery Left:", battery, "minutes")
```

```
else:
```

```
    break
```

```
print("\nMaximum Deliveries:", deliveries)

print("Remaining Battery:", battery, "minutes")
```

OUTPUT;

```
print("Remaining Battery:", battery, "minutes")

... Warehouse Robot Simulation
-----
Delivery 1 | Battery Left: 25 minutes
Delivery 2 | Battery Left: 20 minutes
Delivery 3 | Battery Left: 15 minutes
Delivery 4 | Battery Left: 10 minutes
Delivery 5 | Battery Left: 5 minutes
Delivery 6 | Battery Left: 0 minutes

Maximum Deliveries: 6
Remaining Battery: 0 minutes
```

RUBRICS FOR CALCULATION

Criteria Weightage	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately.	Identifies most constraints with minor gaps.	Partial understanding; misses key constraints.	Fails to identify constraints correctly.
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided
---------------	-----	--	--	-------------------------------	---------------------------